Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates

Leslie N. Smith, Nicholay Topin

arXiv:1708.07120 (2017)

Efficient Models, ИТМО, 2026

О чем статья?

- Статья рассказывает о новом методе изменения LR во время обучения используя большие значения шага, который оказывается лучше классических ступенчатых снижений, при этом улучшая 2 метрики обучения: **скорость и точность**. Этот метод назвали **1cycle**
- В статье на экспериментах показан выигрыш нового метода на задаче обучения нескольких моделей классификации на классических известных CV датасетах
- Кратко можно сказать, что суть метода заключается в **сильном росте LR** от начала обучения до его середины, после чего идет обратное уменьшение до значений, которые были на старте или даже меньше. Логика авторов в том, что:
  - на старте нам важно выбрать правильно направление движения,
  - а после на длинном пологом пути мы можем идти гораздо более широкими шагами, то есть с большим LR,
  - после чего, чтобы точно попасть в наш локальный минимум, шаги снова стоит уменьшить до исходных или еще ниже.

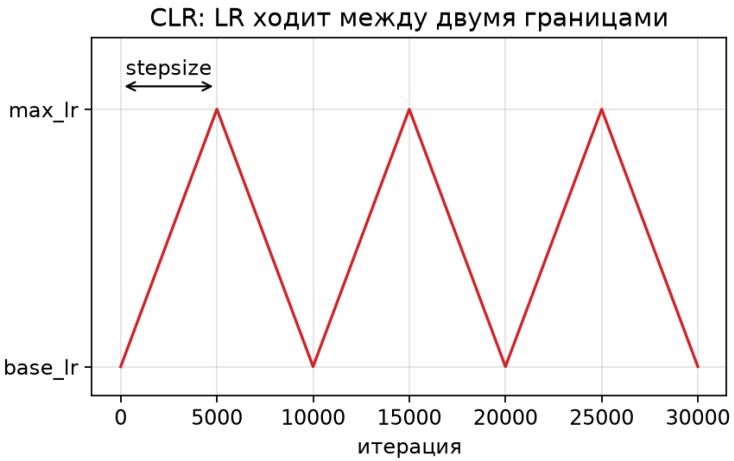
  Используя такой метод, авторы получили точность выше при сильно меньшем количестве итераций, это явление авторы назвали ***super-convergence***

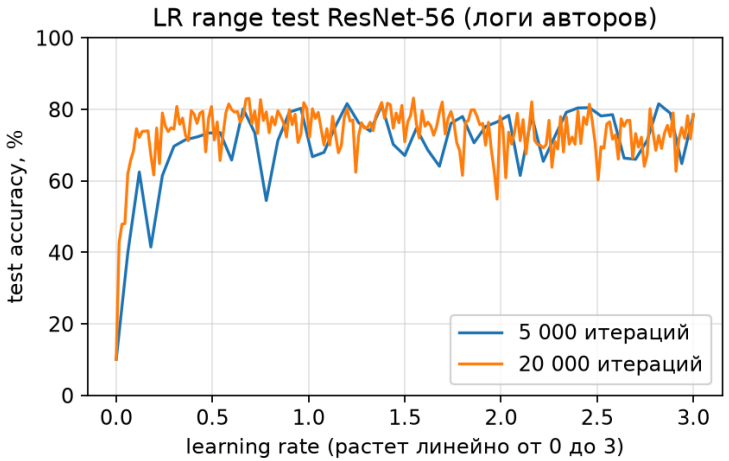
Статья показывает результат метода на экспериментах и без строгих доказательств. Состоит из 3 основных пунктов:

1. Эксперименты — основное: 1cycle против ступенчатого LR на CIFAR-10, на малых данных, на других сетях и на ImageNet
2. Объяснение: большой LR сам регуляризует, поэтому остальную регуляризацию (например, weight decay) надо ослабить. Это тоже показано на экспериментах
3. Формула оценки LR через кривизну — дополнительный аргумент, сами авторы называют ее иллюстрацией

Откуда детали. В тексте статьи постановка экспериментов описана слабо: батч упомянут в сноске, про аугментации ничего нет. Поэтому батч, аугментации, архитектуру и метод изменения LR здесь и дальше я беру из конфигов и логов в репозитории авторов: github.com/lnsmith54/super-convergence

CLR и LR range test



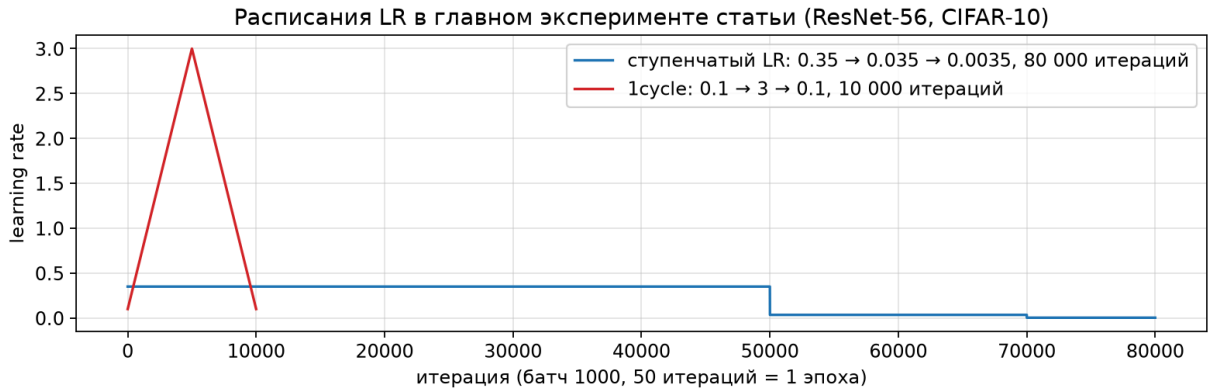


CLR — это циклическое увеличение и уменьшение значений LR на всем промежутке обучения, которое оказалось менее эффективным, по словам авторов, в сравнении с 1cycle, то есть всего 1 полным циклом CLR, после которого LR опускается еще ниже.

LR range test — это тест, который показывает, какое максимальное значение LR можно выбрать и показывает, есть ли у сети потенциал к super-convergence. То есть если при повышении LR точность остается долго высокой, то это хороший признак для того, что в обучении можно использовать 1cycle с очень большим LR.

- CLR (Smith, 2015): LR линейно ходит между `base_lr` и `max_lr`, полцикла — `stepsize` итераций
- LR range test: один короткий прогон, LR растет от 0. Обычно у точности есть пик, его LR берут как `max_lr`, а `base_lr` — в 3–4 раза меньше
- У ResNet-56 пика нет: точность держится на 70–80% вплоть до LR = 3

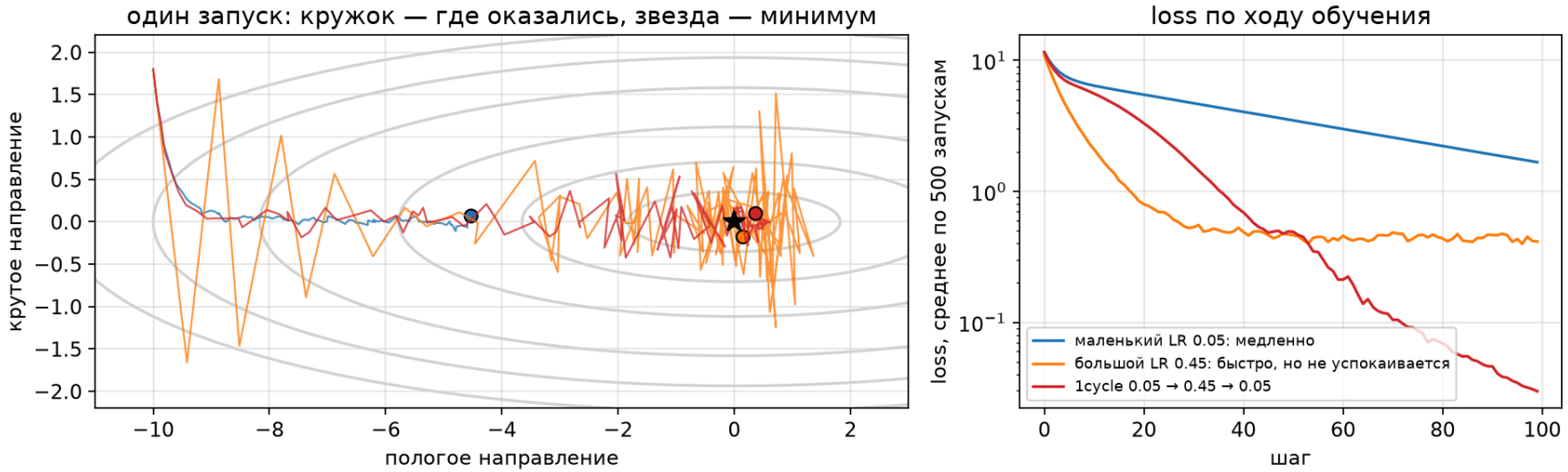
Суть метода 1cycle



1. **Верхняя граница** `max_lr` — по LR range test: самый большой LR, при котором точность еще не падает. У ResNet-56 это 3
2. **Нижняя граница** намного меньше: в экспериментах статьи в 10–40 раз (0.1 → 3 на CIFAR-10, 0.1 → 1 у wide ResNet, 0.01 → 0.1 на MNIST)
3. **Один цикл** вверх-вниз, немного короче всего обучения. На оставшихся итерациях LR опускается еще на несколько порядков
4. **Momentum меняется наоборот с LR:** пока LR растет, momentum падает (0.95 → 0.85), пока LR падает — растет обратно (0.85 → 0.95). Так сделано в Table 2, в тексте статьи это не объясняется
5. **Остальную регуляризацию ослабить:** у DenseNet weight decay 10^{-6} вместо 10^{-4}, на ImageNet $\mathbf{3} \cdot 10^{-6}$ – 10^{-5} вместо 10^{-4} (почему — на слайде 6)

В главном эксперименте на CIFAR-10 авторы взяли упрощенный вариант: чистый треугольник **0.1 → 3 → 0.1 за 10k итераций**, без дальнейшего уменьшения в конце и с постоянным momentum 0.9 (так в конфиге авторов)

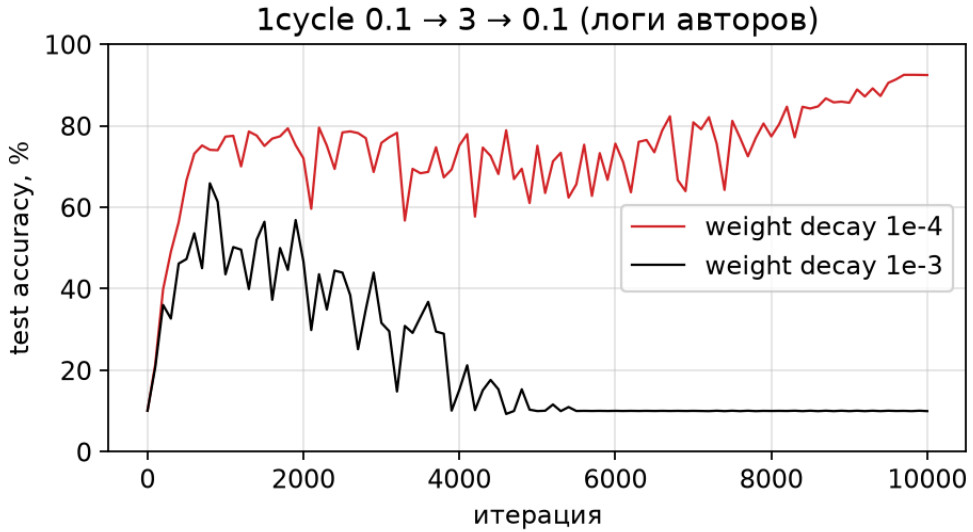
Интуиция авторов: длинная пологая долина (раздел A.1)



- Авторы опираются на картинку из Goodfellow et al. (2014): быстрый спуск, потом долгий путь по пологой долине, в конце узкая впадина, то есть наш локальный минимум
- Отсюда и логика работы 1cycle: малый LR в начале, чтобы пойти в правильную сторону; большой в середине, потому что на пологом дне градиенты маленькие; снова малый в конце, чтобы попасть и остаться в локальном минимуме
- Рисунок — примерная иллюстрация: SGD с шумом на вытянутой параболе. Маленький LR не доходит, большой доходит, но перепрыгивает локальный минимум, 1cycle доходит и остается там

Главная идея: большой LR — это регуляризация (раздел A.2)

- В LR range test при росте LR примерно от 0.2 до 2 train loss растет, а test loss падает (Fig. 8a). Сеть хуже подгоняется под обучающую выборку, но, получается, лучше обобщает данные
- Второй аргумент: с большим LR итоговая точность на тесте выше, а регуляризация по определению — все, что уменьшает ошибку обобщения
- Мысль из статьи: регуляризацию LR надо балансировать с любой иной. Если берем большой LR, то weight decay, dropout и другие надо уменьшать
- Справа на картинке: с weight decay 10^{-3} вместо 10^{-4} сеть скатывается к 10% (Fig. 12b). На ImageNet weight decay снизили до $3 \cdot 10^{-6}$ – 10^{-5}



Почему большой LR регуляризует: шум SGD (раздел A.3)

Шум — это ошибка градиента из-за того, что он посчитан по батчу. Масштаб шума SGD (Smith & Le, 2017):

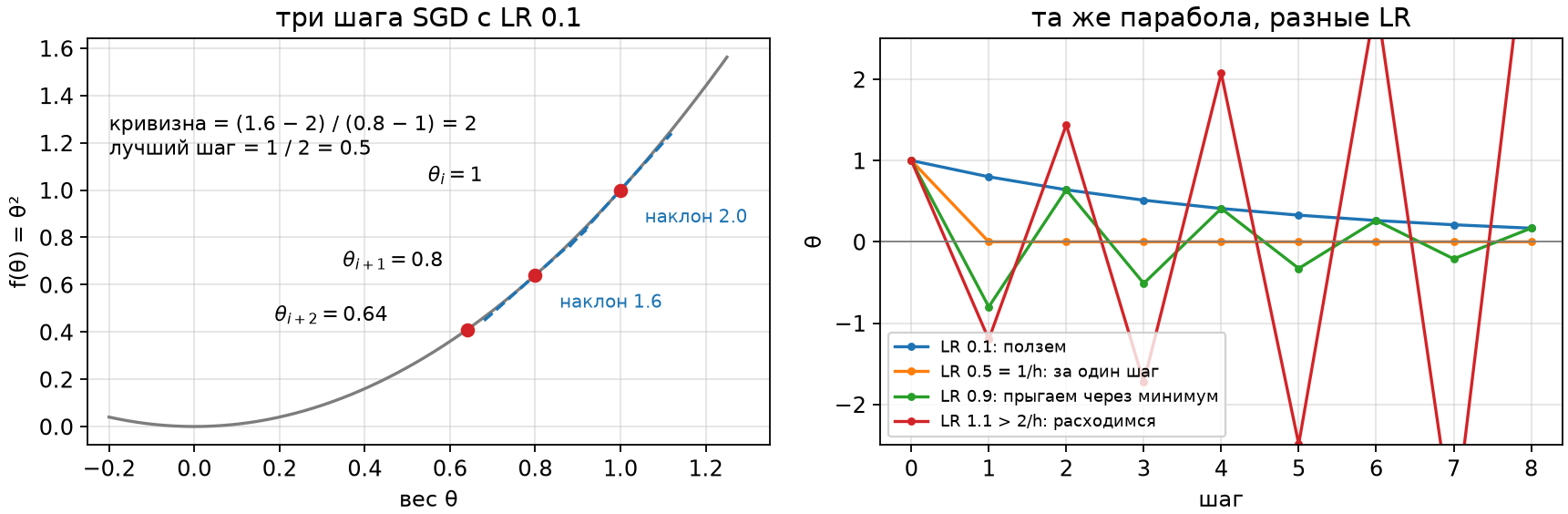
$$g \approx \frac{\epsilon N}{B(1-m)}$$

ϵ — LR, N — размер датасета, B — батч, m — momentum. Больше LR — больше шума, больше батч — меньше. Для CIFAR-10 (N = 50 000, m = 0.9):

Расписание	LR	Батч	Шум g: начало → пик → конец
1cycle статьи	0.1 → 3 → 0.1	1000	50 → **1500** → 50
ступенчатый LR статьи	0.35 → 0.035 → 0.0035	1000	175 → 17.5 → 1.75

- Шум помогает обобщению: отношение LR к батчу управляет шириной минимумов, которые находит SGD (Jastrzębski et al.). Увеличить батч — почти то же, что уменьшить LR (Smith et al., 2017). Авторы добавляют, что шум — тоже регуляризация, и его надо балансировать с остальными видами.
- Keskar et al. советовали начинать с маленького батча, то есть с сильного шума, и потом его убирать. Авторы считают наоборот: начинать с малого шума, дойти до пика и убрать шум в конце, как в 1cycle

Оценка оптимального LR (раздел 4): простой пример



- Слева: парабола $f(\theta) = \theta^2$, три шага SGD с LR 0.1. Вес идет $1 \to 0.8 \to 0.64$, наклон (градиент) в этих точках 2 и 1.6
- Кривизна = изменение наклона / сдвиг веса = $(1.6 - 2) / (0.8 - 1) = 2$, значит лучший шаг $1 / 2 = 0.5$
- Справа проверка: с LR 0.5 минимум за один шаг, с 0.1 идем очень медленно, с 0.9 перепрыгиваем через минимум, с 1.1 вообще беда
- В сети так же, только весов ~0.8 млн и все шумит.

Оценка оптимального LR (раздел 4): формула в общем случае от авторов

Идея. У параболы $f(\theta) = \frac{h}{2}\theta^2$ лучший шаг $\epsilon^* = 1/h$, где θ — вес, а h — кривизна, то есть вторая производная. В примере с прошлого слайда $h = 2$ и $\epsilon^* = 0.5$.

Кривизна по двум точкам. Кривизна — это скорость, с которой меняется градиент: $h \approx (\nabla f(\theta_{i+1}) - \nabla f(\theta_i))/(\theta_{i+1} - \theta_i)$, отсюда $\epsilon^* = 1/h$ (формула 7, идея из Hessian-free методов).

Градиенты через веса. В SGD шаг $\theta_{i+1} = \theta_i - \epsilon\,\nabla f(\theta_i)$, значит $\nabla f(\theta_i) = (\theta_i - \theta_{i+1})/\epsilon$. Подставляем и получаем формулу 8:

$$\epsilon^* = \epsilon\,\frac{\theta_{i+1} - \theta_i}{2\theta_{i+1} - \theta_i - \theta_{i+2}}$$

- $\theta_i,\ \theta_{i+1},\ \theta_{i+2}$ — один и тот же вес на трех итерациях подряд; ϵ — LR, с которым сеть учили; ϵ^* — оценка "оптимального" LR для этого веса
- Числитель — насколько вес сдвинулся за первый шаг, знаменатель — насколько второй шаг отличается от первого. Если шаги почти одинаковые, кривизна маленькая, и ϵ^* получается большим

Проверка на той же параболе: веса $1 \to 0.8 \to 0.64$ при $\epsilon = 0.1$, формула дает $0.1 \cdot \frac{0.8-1}{1.6-1-0.64} = 0.5$ — то же, что мы посчитали по наклонам.

Оценка оптимального LR: что получилось и насколько реально применимо

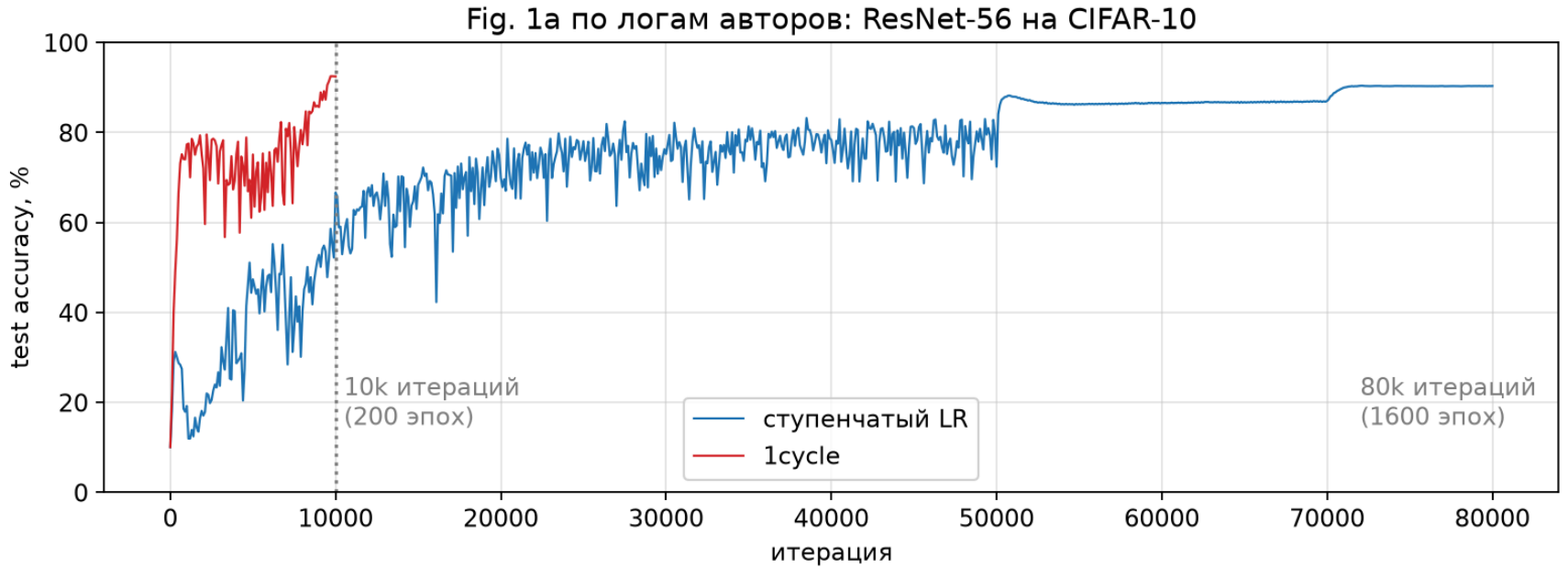
- Формула дает свое число для каждого из ~0.8 млн весов. Чтобы получить одно число на всю сеть, авторы делят сумму модулей всех числителей на сумму модулей всех знаменателей и сглаживают по итерациям: $LR \leftarrow 0.1\,\epsilon^* + 0.9\,LR$
- **Fig. 3a**, первые 300 итераций: оценка 2–6, хотя учили с LR 0.1 (и с CLR 0.1 → 3). Вывод авторов: "оптимальный" LR намного больше обычного, а маленькая кривизна значит широкий плоский минимум
- **Fig. 3b**, 10k итераций: при постоянном LR оценка подскакивает примерно до 3, а потом падает примерно до 0.2. При CLR держится высокой и в конце опускается примерно до 0.5
- Авторы называют это иллюстрацией и пишут в статье, что полную проверку оставляют на будущее
- Важный нюанс: формула выведена для SGD без momentum, а сети, судя по конфигу, учили с momentum 0.9. В примере с параболой с momentum 0.9 веса идут $1 \to 0.8 \to 0.46$, и формула дает $\epsilon^* \approx -0.14$ вместо 0.5
- Градиенты по батчам шумные, а знаменатель — разность двух шумных шагов. И результат зависит от того, что суммировать: в скрипте авторов в сумму попадают еще и накопленные статистики BatchNorm

Как и на чем авторы поставили эксперимент

Из раздела A.4 статьи, конфигов и логов в репозитории авторов:

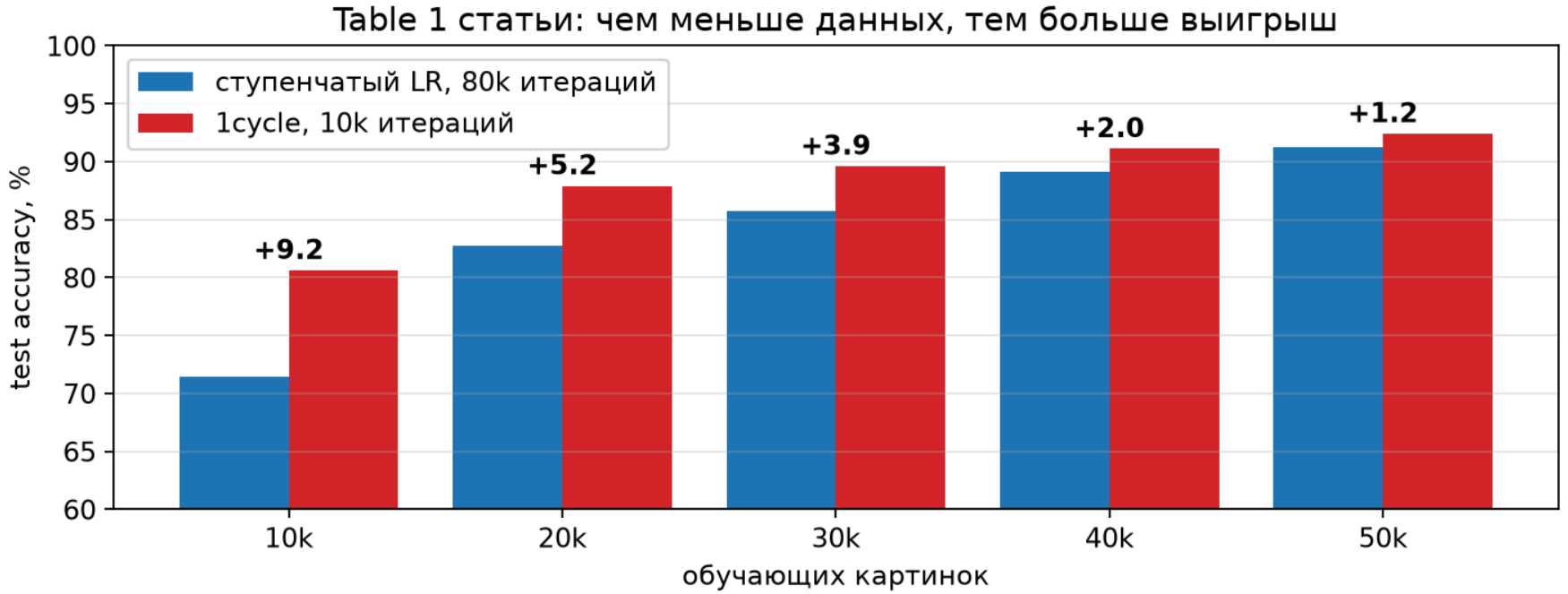
	Авторы (2017)
Сеть	ResNet-56: 3 стадии по 9 блоков, 16 / 32 / 64 канала, ≈ 0.81M параметров. В блоке, где карта уменьшается, свертки не меняют число каналов, shortcut — avg-pool, недостающие каналы дописываются нулями. Настоящая схема есть только в репозитории авторов в файле `Resnet56Cifar.prototxt`, в Table 5 статьи не совсем так
Фреймворк	Caffe
Железо	8× GeForce GTX Titan Black 6ГБ VRAM, часть опытов на 4× Tesla P100
Батч	1000 = 8 GPU × 125, у каждой GPU свои статистики BatchNorm
Аугментация	горизонтальный flip
BN `moving_average_fraction`	0.95 для 1cycle, 0.999 для ступенчатого LR
Итерации и эпохи	50 итераций = 1 эпоха: 10k итераций = 200 эпох, 80k = **1600 эпох**
Время одного прогона по логам	1cycle 10k итераций — **6 ч**, ступенчатый 80k — **49 ч**

Главный результат (Fig. 1a)



- В статье: 1cycle — **92.4%** за 10k итераций, ступенчатый LR — **91.2%** за 80k. В логах из репо авторов 92.44% и 90.32%
- Результат с 1cycle: пока LR большой, точность скачет между 60 и 80%, а весь рост до 92% происходит на спуске LR
- Длина цикла (Fig. 1b): 2k итераций — 89.7%, 6k — уже 92.1%, то есть лучше бейзлайна; 20k — 92.7%

Мало обучающих данных (Table 1)



- Учат на случайной подвыборке CIFAR-10 тем же числом итераций, тестируют на полном тестовом наборе
- По словам авторов, со ступенчатым LR на малых выборках обучение по ходу расходится, а 1cycle учится без проблем. Но это, возможно, не совсем так.
- Вывод авторов: super-convergence тем полезнее, чем меньше размеченных данных

Другие датасеты и ImageNet

Задача	Обычное обучение	1cycle
CIFAR-10, wide ResNet-32	90.3% (800 эпох)	91.9% (100 эпох)
CIFAR-10, DenseNet-40	92.7% (400 эпох)	92.8% (150 эпох)
CIFAR-100, ResNet-56	61.6% (200 эпох)	69.0% (100 эпох)
MNIST, LeNet	99.03% (85 эпох)	99.25% (12 эпох)
ImageNet, ResNet-50	63.7% (100 эпох)	**67.6% (20 эпох)**
ImageNet, Inception-ResNet-v2	67.6% (100 эпох)	**74.0% (20 эпох)**

- В Table 2 батч 512 и уже "полный" 1cycle: вместе с LR циклически меняется momentum (например, 0.95 → 0.85 → 0.95)
- ImageNet: бейзлайн по рецепту Szegedy et al. (LR 0.045, умножается на 0.94 каждые 2 эпохи, weight decay 10^{-4}). 1cycle: LR 0.05 → 1.0 → 0.00005 за 20 эпох, weight decay снижен до $3 \cdot 10^{-6}$ – 10^{-5}. Кривые усреднены всего по двум запускам
- Вывод авторов: эффект не только про ResNet-56 на CIFAR-10. Но везде нужно подбирать регуляризацию заново

Эксперименты из приложения статьи

ResNet-56 на CIFAR-10, цикл 0.1 → 3 → 0.1 за 10k итераций, батч 1000, momentum 0.9; меняется один параметр:

Что меняли	Значение → точность, %
`max_lr` (Table 6)	1.0 → 91.3; 1.5 → 90.9; 2.0 → 91.7; 2.5 → 92.3; **3.0 → 92.4**; 3.5 → 92.1
длина цикла (Table 6)	2k → 89.7; 4k → 91.1; 6k → 92.1; 8k → 91.7; 10k → 92.4; 20k → 92.7
батч (Table 7)	256 → 89.5; 512 → 91.7; **1000 → 92.4**; 1536 → 92.1
momentum (Table 8)	0.80 → 92.1; 0.85 → 91.9; **0.90 → 92.4**; 0.95 → 90.7
weight decay (Fig. 12b)	10^{-3} ломает обучение; 10^{-4}, 10^{-5} и 10^{-6} работают

- Нужен большой `max_lr`, около 3 лучше всего. Цикл подлиннее — в целом лучше. Для коротких циклов авторы уменьшали `moving_average_fraction` у BatchNorm (0.80 для 2k итераций, 0.97 для 20k), иначе статистики для теста не успевают за весами и все начинает ломаться
- Батч побольше лучше: с большим батчем можно взять большой LR.
- Momentum от 0.8 до 0.9 почти одинаков, 0.95 хуже. В "полном" 1cycle momentum меняют обратно по отношению к LR
- Адаптивные оптимизаторы (Fig. 9): с 1cycle super-convergence есть у Nesterov, AdaDelta и AdaGrad, а у Adam нет, как написали авторы.

Спорные места статьи

- **Сравнение в итерациях, а не в эпохах.** Батч 1000 упомянут только в сноске. По конфигам 1cycle — это 200 эпох, а бейзлайн — 1600 эпох
- **Слабые бейзлайны.** ResNet-50 на ImageNet: 63.7% против обычных ~76%
- **Постановка эксперимента не описана в статье.** Батч, аугментации и архитектура есть только в репозитории, в Table 5 не так
- **Слабый бейзлайн в Table 1 из-за BatchNorm.** В логе авторов на 10k картинок ступенчатый LR до 48k-й итерации дает на тесте ~10%, хотя на обучающей выборке почти 100%. С `moving_average_fraction` 0.999 статистики BN не успевают за весами, и сеть ломается только на тесте. Часть выигрыша 1cycle на малых данных может быть из-за этого
- **Оценка оптимального LR** — пример, а не доказанный метод, и он не учитывает momentum

Метод 1cycle сегодня

- 1cycle стал стандартным методом: `torch.optim.lr_scheduler.OneCycleLR` в PyTorch прямо ссылается на эту статью
- LR range test тоже встроен в библиотеки (`lr_find` в fast.ai)
- Схема "разогрев → большой LR → спуск почти до нуля" стала базой. Самый популярный вариант сейчас — разогрев и косинус.

	1cycle статьи	**`OneCycleLR` в PyTorch (по умолчанию)**	**разогрев + косинус**
Подъем LR	линейно 0.1 → 3 за половину обучения	по косинусу от 3/25 = 0.12 до 3 за первые 30%	линейно от 0 до 3 за первые ~5%
Спуск LR	линейно 3 → 0.1	по косинусу почти до нуля ($\approx 1.2 \cdot 10^{-5}$)	по косинусу до 0
Momentum	постоянный 0.9	0.95 → 0.85 → 0.95	постоянный 0.9

Итоги

Статья:

1. Эксперименты: один цикл LR с очень большим максимумом учит ResNet в разы быстрее ступенчатого LR (в итерациях и относительно очень длинного бейзлайна)
2. Главная идея: большой LR — это регуляризация, поэтому остальную регуляризацию, прежде всего weight decay, надо уменьшать
3. Инструмент: LR range test — способ выяснить, можно ли использовать большой LR для этой сети

Я воспроизвел

- 1cycle 0.1 → 3 → 0.1: **92.5 ± 0.2%** (3 seed) против 92.4% в статье, за 8 минут вместо 6 часов
- LR range test, Table 1 для 1cycle, эксперименты из приложения и ResNet-20 сходятся со статьей. А вот оценка LR 2–6 получается, только если сложить веса со статистиками BatchNorm, как в скрипте авторов
- Table 2: DenseNet-40 и MNIST сходятся со статьей, wide ResNet-32 — частично
- `OneCycleLR` и разогрев с косинусом при том же `max_lr` дают ту же точность, что треугольник статьи

Ссылки: статья arXiv:1708.07120 · код авторов github.com/lnsmith54/super-convergence